



1

Progetto A.A. 2023/2024

DIGITAL PERSONAL TRAINER

0280155

Alessandro Lori

Indice

1. Descrizione del Minimondo.....	2
2. Analisi dei Requisiti	3
3. Progettazione concettuale.....	6
4. Progettazione logica	11
5. Progettazione fisica	22
Appendice: Implementazione	Errore. Il segnalibro non è definito.

1. Descrizione del Minimondo

1 Si vuole realizzare un'applicazione per consentire, ai personal trainer di una palestra, di
2 gestire le schede degli esercizi dei propri clienti.

3 Ciascun utente della palestra è associato ad un personal trainer di riferimento. Questi potrà
4 redigere una scheda di allenamento personalizzata per ciascun utente, scegliendo gli esercizi
5 e i macchinari da utilizzare in un insieme definito dal proprietario della palestra. Per ciascun
6 esercizio, il personal trainer indicherà anche il numero di serie e ripetizioni dell'esercizio.

7 Periodicamente, quando viene redatta una nuova scheda di esercizi, la scheda precedente
8 viene archiviata. Questa sarà ancora consultabile dagli utenti, ma essi potranno "interagire"
9 solo con quella corrente.

10 Gli utenti della palestra possono infatti accedere all'applicazione e mostrare quali sono gli
11 esercizi che devono svolgere in una sessione di allenamento. Il sistema mostrerà l'esercizio
12 e la serie corrente, permettendo all'utente di contrassegnare un dato esercizio come
13 completato. L'atleta ha la possibilità di saltare un esercizio, anche se cominciato, e passare
14 al successivo.

15 I personal trainer possono generare un report che mostra, per tutti i clienti a loro assegnati,
16 quanti sono gli allenamenti sostenuti in un intervallo temporale richiesto, qual è la
17 percentuale di completamento delle schede di allenamento in ogni sessione di allenamento e
18 quanto tempo è durato ciascun allenamento.

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
3	Questi	Il personal trainer	Termine più chiaro
8	Questa	La scheda archiviata	Termine più chiaro
8	Essi	Gli utenti	Termine più chiaro
9	Quella	La scheda	Termine più chiaro
10	Infatti	“ “	Termine non necessario
13	L'atleta	L'utente	Termine più chiaro

Specifica disambiguata

Si vuole realizzare un'applicazione per consentire, ai personal trainer di una palestra, di gestire le schede degli esercizi dei propri clienti.

Ciascun utente della palestra è associato ad un personal trainer di riferimento. Il personal trainer potrà redigere una scheda di allenamento personalizzata per ciascun utente, scegliendo gli esercizi e i macchinari da utilizzare in un insieme definito dal proprietario della palestra. Per ciascun esercizio, il personal trainer indicherà anche il numero di serie e ripetizioni dell'esercizio.

Periodicamente, quando viene redatta una nuova scheda di esercizi, la scheda precedente viene archiviata. La scheda archiviata sarà ancora consultabile dagli utenti, ma gli utenti potranno “interagire” solo con la scheda corrente.

Gli utenti della palestra possono accedere all'applicazione e mostrare quali sono gli esercizi che devono svolgere in una sessione di allenamento. Il sistema mostrerà l'esercizio e la serie corrente, permettendo all'utente di contrassegnare un dato esercizio come completato. L'utente ha la possibilità di saltare un esercizio, anche se cominciato, e passare al successivo.

I personal trainer possono generare un report che mostra, per tutti i clienti a loro assegnati, quanti sono gli allenamenti sostenuti in un intervallo temporale richiesto, qual è la percentuale di completamento delle schede di allenamento in ogni sessione di allenamento e quanto tempo è durato ciascun allenamento.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Personal Trainer	Responsabile della creazione di schede e report		Schede, Utenti, Report
Scheda Archiviata	Lista di esercizi che il cliente può solo visionare		Clienti, Utenti
Utenti	Persone che usufruiscono dei servizi della palestra		Scheda, Personal Trainer, Serie
Scheda	Lista di esercizi con la quale il cliente può interagire	Scheda corrente	Personal Trainer, Utenti, Serie
Serie	Quantitativo di ripetizioni per ogni esercizio		Scheda, Utenti, Sessione di Allenamento
Sessione di Allenamento	Singola attività dell'utente nella palestra svolgendo o saltando esercizi		Utente, Scheda, Sessione di Allenamento
Report	Resoconto degli allenamenti di ogni cliente associato ad un personal trainer		Personal Trainer, Utenti, Sessione di Allenamento

Raggruppamento dei requisiti in insiemi omogenei

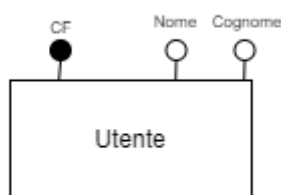
Fraasi relative a PERSONAL TRAINER
Ogni Personal Trainer è identificato da un ID equivalente al Codice Fiscale (CF) e sono responsabili della gestione delle schede degli utente.
Fraasi relative a UTENTE
Ogni utente è identificato dal proprio Codice Fiscale (CF). Ogni utente è associato ad un Personal Trainer. L'utente può interagisce con la scheda attuale e visionare tutte le schede che la precedono.
Fraasi relative a SCHEDA
Ogni Scheda è identificata dalla data di emissione e dall'utente al quale è associate. La scheda può essere attiva o archiviata.
Fraasi relative a SCHEDA ATTIVA
La Scheda Attiva è il tipo di scheda con la quale l'utente interagisce visionando ed eseguendo gli esercizi indicate.
Fraasi relative a SCHEDA ARCHIVIATA
La Scheda Archiviata è il tipo di scheda che l'utente può visionare in sola lettura. Di tale scheda è anche nota la data in cui è scaduta ed è passata dal tipo attiva ad archiviata.
Fraasi relative a SERIE
Le Serie rappresentano il numero di volte in cui l'esercizio indicato sulla scheda deve essere eseguito
Fraasi relative a SESSIONE DI ALLENAMENTO
Ogni Sessione di Allenamento è tracciata per ogni utente e identificata sia dall'utente che dalla data di esecuzione, inoltre sappiamo l'orario di inizio e di fine.
Fraasi relative a REPORT
Ogni personal trainer può generare un Report nel quale sono contenute, per un dato lasso di tempo, le statistiche di allenamento di ogni utente a lui associato

3. Progettazione concettuale

Costruzione dello schema E-R

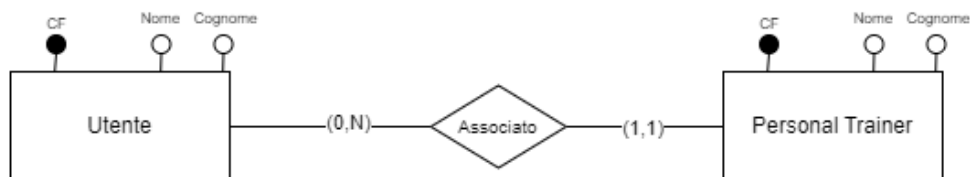
Organizziamo la progettazione concettuale della nostra base dati adottando la strategia inside-out, che consiste nello sviluppare prima i concetti più importanti (primari) in bottom-up. Gli altri concetti secondari verranno successivamente sviluppati per collegamento ai concetti primari.

-Tutto ciò che avviene nel mini-mondo gira attorno all'entità Utente che rappresenta l'usufruttore dei servizi erogati dalla palestra. Di questo teniamo traccia del nome, cognome e codice fiscale. Ottenendo:



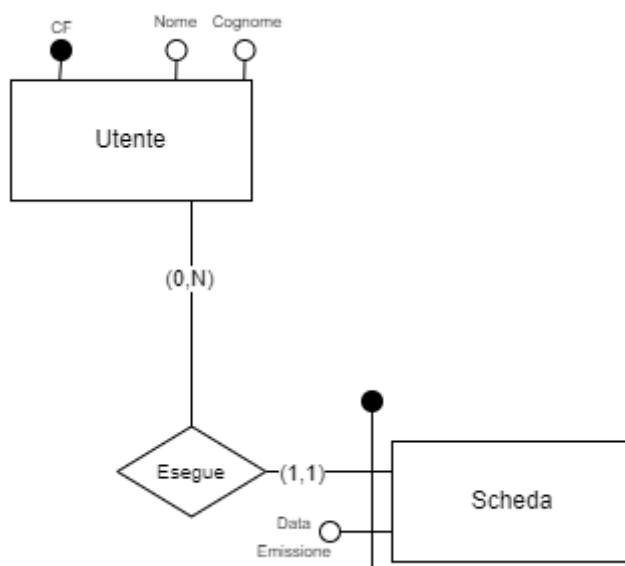
-All'Utente è associato un Personal Trainer. Di ogni Personal Trainer si tiene traccia di nome, cognome e codice fiscale. Cliente e Personal Trainer sono collegati dalla relazione "Associato" di tipo "uno a molti" in quanto ogni ad ogni Cliente corrisponde al più un Personal Trainer mentre ad ogni Personal Trainer corrispondono al più n Clienti.

Ottenendo:



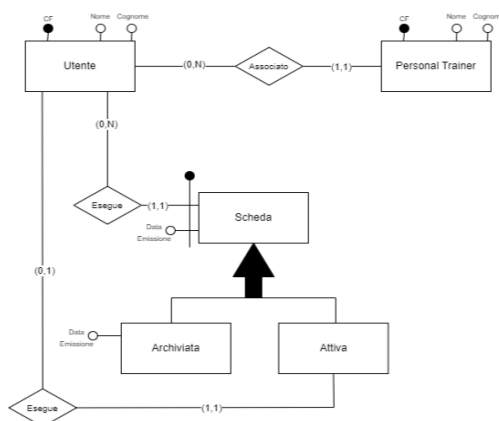
-Il Cliente esegue una data Scheda. Di questa teniamo traccia della data di emissione e dell'Utente ad essa destinata. Cliente e Scheda sono collegate tramite la relazione "Esegue" di tipo "uno a molti" in quanto ogni Utente può eseguire al più una Scheda alla volta mentre ogni Scheda può essere eseguita da più Utenti.

Ottenendo:



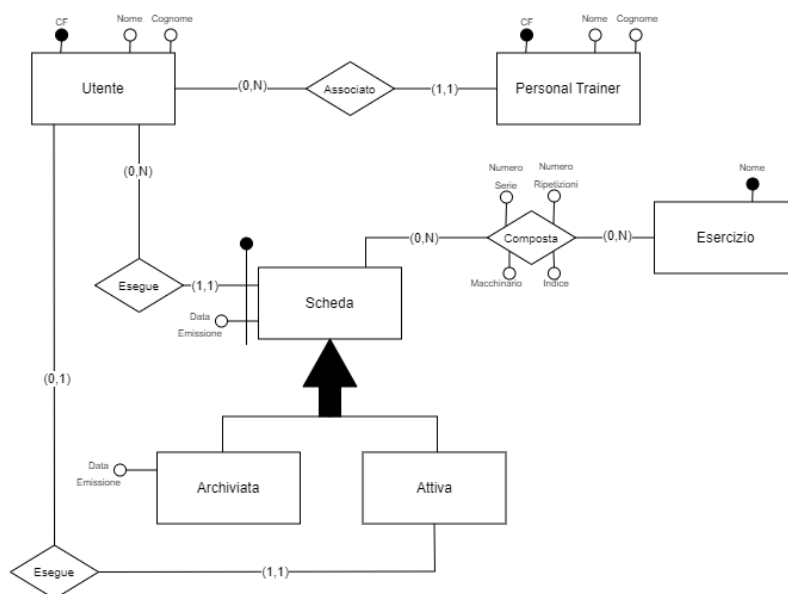
-Come da specifica le Schede sono di due tipi: Scheda Archiviata e Scheda Attiva. La Scheda Attiva è la scheda che attualmente l'utente segue e svolge fino all'emissione di una nuova scheda ed è quella con la quale l'Utente interagisce. Le Schede Archivate sono tutte le schede antecedenti all'attuale Scheda Attiva, caratterizzate dalla data di scadenza e sono le schede che l'utente può visionare in sola lettura.

Ottenendo:



-La scheda che l'Utente segue è popolata dall'entità Esercizi. Di questa teniamo traccia del nome ed è collegata all'entità Scheda dalla relazione "Composta" di tipo "molti a molti" attraverso la quale possiamo tenere traccia del numero di ripetizioni, numero di serie, macchinario usato e indice di posizione nella Scheda per ogni Esercizio.

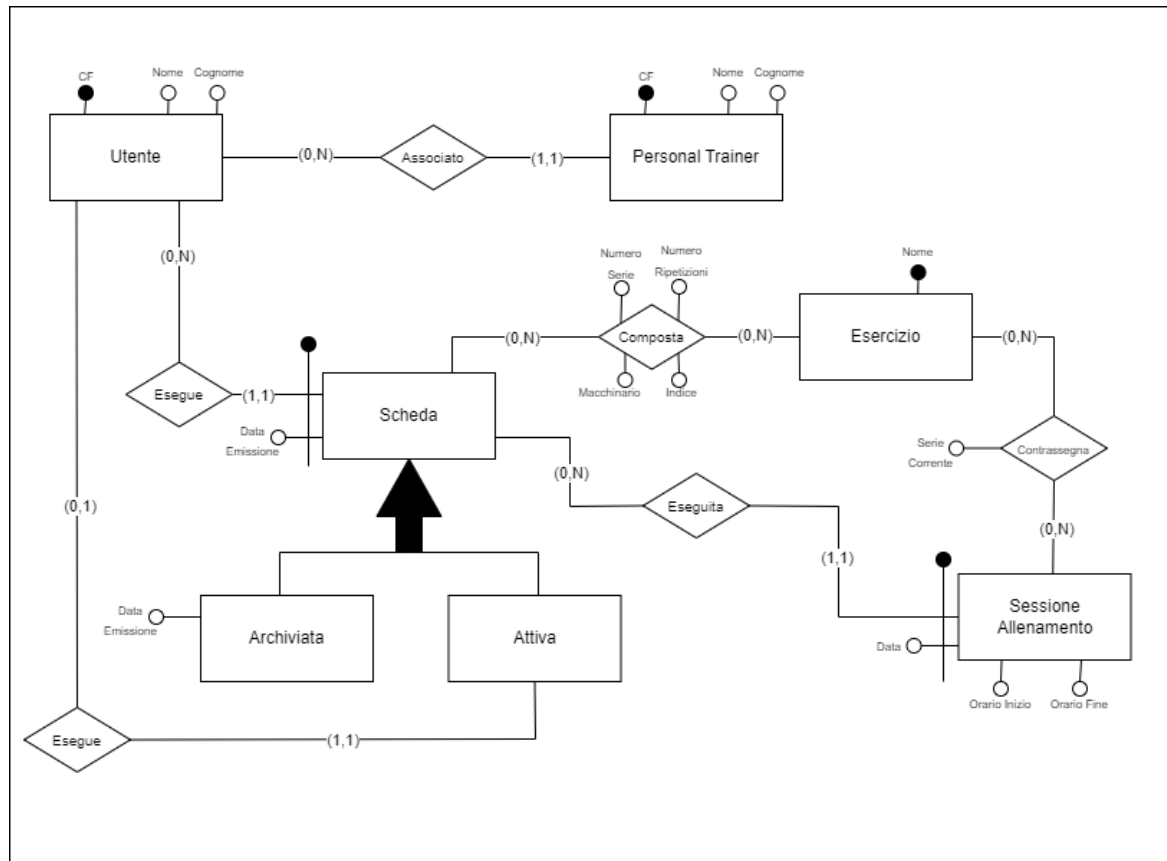
Ottenendo:



-Infine Sessione di Allenamento è l'entità con la quale teniamo traccia degli orari di allenamento (orario inizio, orario fine) e della data correlate alla Scheda che l'Utente ha eseguito in quella giornata. Sessione di Allenamento e Scheda sono infatti collegati attraverso la relazione "Eseguita" di tipo "uno a molti" in quanto durante una Sessione di Allenamento può essere eseguita una sola Scheda mentre una stessa Scheda può essere eseguita per più Sessioni di Allenamento. Sessione di Allenamento è inoltre collegata all'entità Esercizi attraverso la relazione "Contrassegna" di tipo "molti a molti" che permette all'Utente di saltare o evidenziare come completati gli esercizi tenendo traccia della serie corrente.

Otteniamo così l'integrazione finale.

Integrazione finale



Regole aziendali

- 1) Il numero di Serie Corrente deve essere minore o uguale al Numero di Serie.
- 2) Ogni Esercizio eseguito o saltato deve far parte della Scheda Attiva dell'utente.
- 3) Ogni Esercizio ha una posizione unica all'interno della Scheda.
- 4) Ogni nuova Scheda Attiva è preceduta dall'archiviazione della Scheda precedente.
- 5) Ogni Scheda Attiva deve essere popolata da Esercizi
- 6) Una Scheda Attiva non è modificabile dopo la sua creazione

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Utente	Usufruitore dei servizi della palestra	Nome, Cognome	CF (codice fiscale)
Personal Trainer	Gestore dell'allenamento degli Utenti	Nome, Cognome	CF
Scheda	Lista di esercizi che l'Utente svolge durante una Sessione di Allenamento		Data Emissione, Utente
Scheda Archiviata	Lista di esercizi svolti in passato dall'utente disponibili in sola lettura	Data Scadenza	Scheda
Scheda Attiva	Lista di esercizi che l'Utente svolge attualmente completando o saltando		Scheda
Esercizi	Elementi di attività fisica presenti nella Scheda dell'Utente		Nome
Sessione di Allenamento	Periodo di tempo durante il quale l'Utente svolge gli Esercizi indicati sulla Scheda in palestra	Orario Inizio, Orario Fine	Data, Scheda

4. Progettazione logica

Volume dei dati

- 1) Si assume un periodo di validità dei dati di 3 anni, dopo i quali tutti i dati vengono definitivamente eliminati dal database
- 2) Si assume che il numero di Utenti già abbonati che utilizzano il sistema sia 500
- 3) Si assume che ogni anno il numero di nuovi Utenti abbonati sia di 100
- 4) Si assume che il numero di Personal Trainer già assunti che utilizzano il sistema sia di 25
- 5) Si assume che ogni anno il numero di Personal Trainer assunti sia di 5
- 6) Si assume che ogni Scheda abbia in media 8 Esercizi
- 7) Si assume che ogni Esercizio venga eseguito in media per 4 Serie
- 8) Si assume che durante ogni Sessione di Allenamento l'Utente svolga circa l'80% degli Esercizi sulla Scheda
- 9) Si assume la visualizzazione della Scheda Attiva in media di 2 volte per Sessione di Allenamento
- 10) Si assume che la visualizzazione della Scheda Archiviata venga fatta dal 10% degli utenti in media ogni anno
- 11) Si assume la disponibilità immediata di 35 Macchinari
- 12) Si assume l'acquisto di 2 Macchinari nuovi ogni anno
- 13) Gli Utenti possono eseguire al più una Sessione di Allenamento al giorno e si assume che ci saranno in media 3 Sessioni di Allenamento a settimana per Utente
- 14) Ogni Scheda Archiviata comporta una nuova Scheda Attiva

Concetto nello schema	Tipo ¹	Volume atteso
Utenti	E	800

¹ Indicare con E le entità, con R le relazioni

Personal Trainer	E	40
Scheda	E	15600
Scheda Attiva	E	800
Scheda Archiviata	E	14800
Esercizi	E	41
Sessione di Allenamento	E	405600
Associato	R	800
Esegue	R	15600
Eseguita	R	405600
Interagisce	R	15600
Compone	R	124800
Contrassegna	R	2595840

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
AM1	Nuovo Cliente	100/anno
AM2	Nuovo Personal Trainer	5/anno
AM3	Aggiungi Esercizio	2/anno
U1	Inizio Allenamento	371/giorno
U2	Fine Allenamento	371/giorno
U3	Esegui Esercizio	2371/giorno
U4	Visualizza Scheda Attiva	741/giorno
U5	Visualizza Scheda Archiviata	37/giorno
U6	Crea Scheda	14/giorno
U7	Archivia Scheda	14/giorno
U8	Aggiungi Serie	57/giorno
U9	Aggiungi Esercizio	144/giorno
U10	Visualizza Utente	20/giorno
U11	Genera Report	6/giorno

L1	Registrazioni	105/anno
L2	Login	391/ANNO

Costo delle operazioni

Si suppone, in questa fase di progettazione, che il costo di scrittura di un dato sia il doppio del costo in lettura.

AM1 – Nuovo Utente:

- Utente: 800(W)
- Associato: 800(W)
- Personal Trainer: 40(L)

Costo Totale: 3240

AM2 – Nuovo Personal Trainer:

- Personal Trainer: 40(W)

Costo Totale: 80

AM3 – Aggiungi Esercizio:

- Esercizio: 41(W)

Costo Totale: 82

U1 – Inizio Allenamento:

- Utente: 800(L)
- Interagisce: 800(L)
- Scheda Attiva: 800(L)
- Eseguita: 800(W)
- Sessione di Allenamento: 800(W)

Costo Totale: 5600

U2 – Fine Allenamento:

- Utente: 800(L)
- Interagisce: 800(L)
- Scheda Attiva: 800(L)
- Eseguita: 800(W)
- Sessione di Allenamento: 800(W)

Costo Totale: 5600

U3 – Esegui Esercizio:

- Utente: 800(L)
- Interagisce: 800(L)
- Scheda Attiva: 800(L)
- Eseguita: 800(L)
- Sessione di Allenamento: 800(L)
- Contrassegna Esercizio: 6400(W)

Costo Totale: 16000

U4 – Visualizza Scheda Attiva:

- Utente: 800(L)
- Interagisce: 800(L)
- Scheda Attiva: 800(L)

Costo Totale: 2400

U5 – Visualizza Schede Archivate:

- Utente: 800(L)
- Esegue: 14400(L)
- Scheda Archiviata: 14400(L)

Costo Totale: 29600

U6 – Crea Scheda:

- Personal Trainer: 40(L)
- Associato: 800(L)
- Utente: 800(L)
- Interagisce: 800(L)
- Scheda Attiva: 800(W)
- Esegue: 800(W)

Costo Totale: 5600

U7 – Archivia Scheda:

- Personal Trainer: 40(L)
- Associato: 800(L)
- Utente: 800(L)
- Interagisce: 800(L)
- Scheda Attiva: 800(L)

- Scheda Archiviata: 800(W)

Costo Totale: 4840

U8 – Aggiungi Serie:

- Personal Trainer: 40(L)

- Associato: 800(L)

- Utente: 800(L)

- Interagisce: 800(L)

- Scheda Attiva: 800(L)

- Eseguita: 800(L)

- Sessione di Allenamento: 800(L)

- Contrassegna: 25600(W)

Costo Totale: 56040

U9 – Aggiungi Esercizio:

- Personal Trainer: 40(L)

- Associato: 800(L)

- Utenti: 800(L)

- Interagisce: 800(L)

- Scheda Attiva: 800(L)

- Composizione: 6400(W)

Costo Totale: 16040

U10 – Visualizza Utente:

- Personal Trainer: 40(L)

- Associato: 800(L)

- Utente: 800(L)

Costo Totale: 1640

U11 – Genera Report:

- Personal Trainer: 40(L)
- Associato: 800(L)
- Utente: 800(L)
- Esegue: 800(L)
- Scheda: 800(L)
- Composizione: 6400(L)
- Contrassegna: 25600(L)

Costo Totale: 35240

L1 – Registrazione:

- Personal Trainer: 40(L)
- Utente: 800(L)

Costo Totale: 840

L2 – Login:

- Personal Trainer: 40(L)
- Utente: 800(L)

Costo Totale: 840

Ristrutturazione dello schema E-R

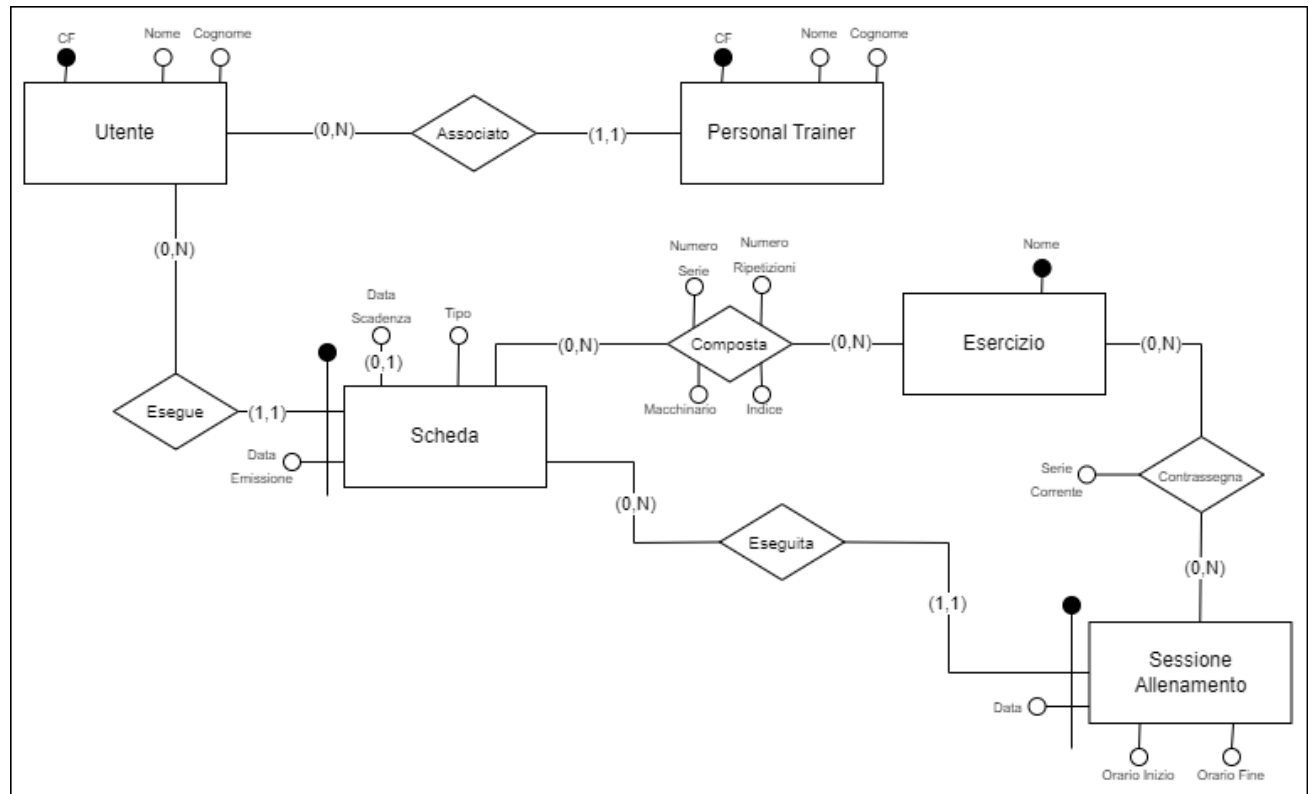
Eliminazione delle Generalizzazioni

Lo schema ER presenta la generalizzazione con Scheda padre e Archiviata e Attiva come figli eliminabile in fase di ristrutturazione. Per poter ristrutturare correttamente questa generalizzazione abbiamo a disposizione 3 modi:

- 1) Portare il padre nei figli dato che i figli supportano operazioni diverse che evidenziano la differenza fra le due entità. Utilizzando questo metodo andremmo a gravare pesantemente sui costi in memoria poiché si andrebbe a duplicare tutte le associazioni che essendo precedentemente legate al padre passerebbero ai figli.
- 2) Eliminare la generalizzazione e sostituirla con associazioni tra padre e figlio. Anche questa soluzione comporterebbe un ingente spreco di memoria in quanto bisognerebbe tenere traccia di due nuove relazioni. In particolare questa soluzione andrebbe a gravare sulle operazioni di creazione e archiviazione scheda (rispettivamente le operazioni U6, U7) aggiungendo due nuove operazioni di lettura o scrittura prima di poter toccare il dato interessato.
- 3) Accorpate i figli all'interno del padre creando così un'unica entità. Per applicare questa soluzione dobbiamo mantenere le unicità dei figli, in questo caso l'entità figlio Scheda Archiviata ha un attributo (Data Scadenza) che non è presente nel secondo figlio Scheda Attiva. Per ovviare a questo problema e poter inoltre distinguere una Scheda Archiviata da una Attiva possiamo aggiungere un attributo denominato "Tipo" di tipo enum che indicherà se la scheda è "attiva" o "scaduta" e aggiungere all'entità accorpata Scheda l'attributo "Data Scadenza" che sarà multivalore (0,1) per far sì che la Scheda Attiva possa avere tale valore impostato a null.

La terza opzione è la più efficiente, non aggiunge ridondanze o grandi quantità di memoria occupata e risolve la ristrutturazione della generalizzazione con la sola aggiunta di 2 attributi.

Otteniamo dunque lo schema ristrutturato in tale modo:



Traduzione di entità e associazioni

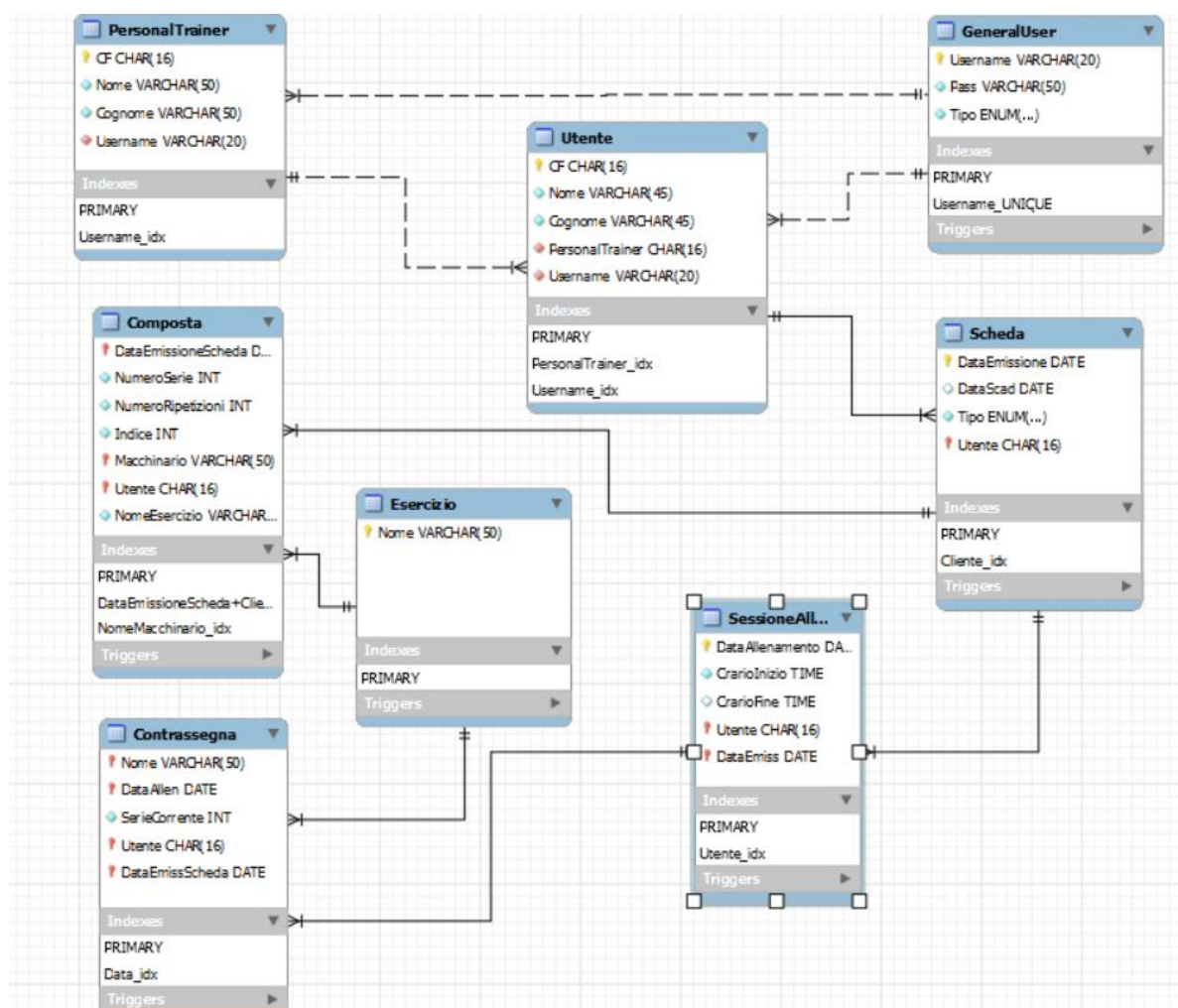
Partendo dallo schema ristrutturato costruiamo le seguenti relazioni per il modello relazionale.

- PersonalTrainer(CF, Nome, Cognome)
- Utente(CF, Nome, Cognome, PersonalTrainer)
 - $\text{Utente}(\text{PersonalTrainer}) \subseteq \text{PersonalTrainer}(\text{CF})$
- Scheda(DataEmissione, Utente, DataScadenza*, Tipo)
 - $\text{Scheda}(\text{Utente}) \subseteq \text{Utente}(\text{CF})$
- Esercizio(Nome)
- SessioneAllenamento(Data, Scheda, OrarioInizio, OrarioFine)
 - $\text{SessioneAllenamento}(\text{Scheda}) \subseteq \text{Scheda}(\text{Utente}, \text{DataEmissione})$
- Composta(Scheda, Esercizio, Macchinario, Indice, NumeroRipetizioni, NumeroSerie)
 - $\text{Composta}(\text{Scheda}) \subseteq \text{Scheda}(\text{Utente}, \text{DataEmissione})$

- Composta(Esercizio) $\subseteq$ Esercizio(Nome)
- Contrassegna(Esercizio, SessioneAllenamento, SerieCorrente)
 - Contrassegna(Esercizio) $\subseteq$ Esercizio(Nome)
 - Contrassegna(SessioneAllenamento) $\subseteq$ SessioneAllenamento(Data, Scheda)

I valori con * rappresentano valori che possono accettare il valore nullo

Rappresentazione grafica del modello:



Normalizzazione del modello relazionale

1NF

Ricordiamo che una relazione soddisfa la 1NF se:

- 1) È presente la chiave primaria
- 2) Non ci sono attributi ripetuti
- 3) Gli attributi non sono strutture complesse

Le tabelle fornite soddisfano la prima forma normale in quanto ogni colonna contiene un singolo valore, non ci sono ripetizioni o gruppi di colonne e le chiavi esterne collegano correttamente le tabelle.

2NF

Ricordiamo che una relazione è in 2NF se la chiave scelta non contiene chiavi più piccole (ovvero la chiave scelta è effettivamente una chiave e non una superchiave).

In tutte le relazioni le chiavi non contengono chiavi più piccole dunque sono minimali.

3NF

Una relazione è in 3NF se per ogni dipendenza funzionale $X \rightarrow A$ vale una delle seguenti:

- 1) X è superchiave della relazione
- 2) A appartiene ad almeno una chiave

Nel nostro caso questo vale per tutte le relazioni.

5. Progettazione fisica

Utenti e privilege

Sono previsti 4 ruoli:

1) Login:

-Grant in esecuzione su L1.

2) Utente:

-Grant in esecuzione su operazioni U1-U5

3) PersonalTrainer:

-Grant in esecuzione su operazioni U6-U11

4) Segreteria:

-Grant in esecuzione su operazioni AM1-AM3

Strutture di memorizzazione

Tabella <GeneralUser>		
Attributo	Tipo di dato	Attributi ²
Username	VARCHAR(20)	PK, NN, UQ
Password	VARCHAR(50)	NN
Tipo	ENUM('pt', 'utente', 'segreteria')	NN

Tabella <Utente>

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Attributo	Tipo di dato	Attributi ³
CF	CHAR(16)	PK, NN
Nome	VARCHAR(45)	NN
Cognome	VARCHAR(45)	NN
PersonalTrainer	CHAR(16)	NN
Username	VARCHAR(16)	NN

Tabella <PersonalTrainer>		
Attributo	Tipo di dato	Attributi ⁴
CF	CHAR(16)	PK, NN
Nome	VARCHAR(45)	NN
Cognome	VARCHAR(45)	NN
Username	VARCHAR(16)	NN

Tabella <Scheda>		
Attributo	Tipo di dato	Attributi ⁵
DataEmissione	DATE	PK, NN
DataScad	DATE	
Tipo	ENUM('scaduta', 'attiva')	NN
Utente	CHAR(16)	PK, NN

Tabella <Composta>

³ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁴ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁵ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Attributo	Tipo di dato	Attributi ⁶
DataEmissioneScheda	DATE	PK, NN
NumeroSerie	INT	NN
NumeroRipetizioni	INT	NN
Indice	INT	NN
Macchinario	VARCHAR(50)	PK, NN
Utente	CHAR(16)	PK, NN
NomeEsercizio	VARCHAR(50)	NN

Tabella <SessioneAllenamento>		
Attributo	Tipo di dato	Attributi ⁷
DataAllenamento	DATE	PK, NN
OrarioInizio	TIME	NN
OrarioFine	TIME	
Utente	CHAR(16)	PK, NN
DataEmiss	DATE	PK, NN

Tabella <Contrassegna>		
Attributo	Tipo di dato	Attributi ⁸
Nome	VARCHAR(50)	PK, NN
DataAllen	DATE	PK, NN
SerieCorrente	INT	NN
Utente	CHAR(16)	PK, NN
DataEmissioneScheda	DATE	PK, NN

⁶ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁷ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

⁸ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Tabella <Esercizio>		
Attributo	Tipo di dato	Attributi ⁹
Nome	VARCHAR(50)	PK, NN

Indici

Si omettono gli indici di tipo PRIMARY per le chiavi e INDEX per le foreign key, in quanto generati automaticamente.

Trigger

GeneralUser_BEFORE_INSERT

Trigger creato per controllare se lo Username di un nuovo utente che si vuole inserire in GeneralUser è già presente nel database

```
CREATE DEFINER = CURRENT_USER TRIGGER `mydb`.`GeneralUser_BEFORE_INSERT`  
BEFORE INSERT ON `GeneralUser` FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE username_count INT;
```

```
    -- Controlla se l'username esiste già
```

```
    SELECT COUNT(*)
```

```
    INTO username_count
```

```
    FROM GeneralUser
```

```
    WHERE Username = NEW.Username;
```

⁹ PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

```
-- Se esiste già, ritorna un messaggio di errore

IF username_count > 0 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Utente già esistente';

END IF;

END
```

Scheda_BEFORE_INSERT

Trigger creato per controllare l'esistenza di una Scheda Attiva di un determinato utente al momento della creazione di una stessa per far rispettare il fatto che non possono esistere due Schede Attive contemporaneamente.

```
CREATE DEFINER = CURRENT_USER TRIGGER `mydb`.`Scheda_BEFORE_INSERT`
BEFORE INSERT ON `Scheda` FOR EACH ROW

BEGIN

    DECLARE count_active_schedules INT;

    -- Controlla se esiste già una scheda attiva per lo stesso utente

    SELECT COUNT(*)

    INTO count_active_schedules

    FROM Scheda

    WHERE Utente = NEW.Utente

    AND DataScad is null; -- Verifica se la data di scadenza è successiva o uguale a oggi
```

```
-- Se esiste una scheda attiva, solleva un errore

IF count_active_schedules > 0 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Esiste già una scheda attiva per questo utente.';

END IF;

END
```

Composta_BEFORE_INSERT

Trigger creato per controllare l'esistenza di un esercizio con la stessa posizione nell'indice per la stessa scheda con un nuovo esercizio in fase di inserimento.

```
CREATE DEFINER = CURRENT_USER TRIGGER `mydb`.`Composta_BEFORE_INSERT`
BEFORE INSERT ON `Composta` FOR EACH ROW

BEGIN

DECLARE count_indices INT;

-- Controlla se esiste già un esercizio con lo stesso indice per la stessa scheda

SELECT COUNT(*)

INTO count_indices

FROM Composta

WHERE DataEmissioneScheda = NEW.DataEmissioneScheda

AND Utente = NEW.Utente

AND Indice = NEW.Indice;

-- Se esiste un esercizio con lo stesso indice, solleva un errore

IF count_indices > 0 THEN
```

```
SIGNAL SQLSTATE '45000'
```

```
SET MESSAGE_TEXT = 'Esiste già un esercizio con lo stesso indice per questa scheda di esercizi.';
```

```
END IF;
```

```
END
```

SessioneAllenamento_BEFORE_INSERT

Trigger creato per controllare che non venga iniziata una sessione di allenamento nell'orario di chiusura della palestra.

```
CREATE          DEFINER          =          CURRENT_USER          TRIGGER  
`mydb`.`SessioneAllenamento_BEFORE_INSERT` BEFORE INSERT ON `SessioneAllenamento`  
FOR EACH ROW
```

```
BEGIN
```

```
    IF HOUR(CURRENT_TIME()) < 7 OR HOUR(CURRENT_TIME()) > 23 THEN
```

```
        SIGNAL SQLSTATE '45007'
```

```
        SET MESSAGE_TEXT = 'Puoi iniziare una sessione solo tra le 7:00 e le 23:00';
```

```
    END IF;
```

```
END
```

SessioneAllenamento_BEFORE_UPDATE

Trigger creato per controllare che non venga aggiornata una sessione di allenamento nell'orario di chiusura della palestra.

```
CREATE DEFINER = CURRENT_USER TRIGGER
`mydb`.`SessioneAllenamento_BEFORE_UPDATE` BEFORE UPDATE ON
`SessioneAllenamento` FOR EACH ROW

BEGIN

    IF HOUR(CURRENT_TIME()) < 7 OR HOUR(CURRENT_TIME()) > 23 THEN

        SIGNAL SQLSTATE '45008'

        SET MESSAGE_TEXT = 'Puoi chiudere una sessione solo tra le 7:00 e le 23:00';

    END IF;

END
```

Contrassegna_BEFORE_INSERT

Trigger creato per controllare se l'esecuzione di un esercizio è già presente nella sessione di allenamento.

```
CREATE DEFINER = CURRENT_USER TRIGGER `mydb`.`Contrassegna_BEFORE_INSERT`
BEFORE INSERT ON `Contrassegna` FOR EACH ROW

BEGIN

    DECLARE count_existing_executions INT;

    -- Controlla se l'esecuzione dell'esercizio è già presente nella sessione di allenamento

    SELECT COUNT(*)

    INTO count_existing_executions

    FROM Contrassegna

    WHERE DataAllen = NEW.DataAllen
```

```
AND Nome = NEW.Nome;
```

```
-- Se l'esecuzione è già presente, solleva un errore
```

```
IF count_existing_executions > 0 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Esecuzione dell'esercizio per questa sessione già presente nel database.';
```

```
END IF;
```

```
END
```

EsercizioInesist_BEFORE_INSERT_1

Trigger creato per controllare l'esistenza di un esercizio all'interno della scheda nel momento in cui l'utente sta andando ad eseguire tale esercizio e avvisare nel caso in cui questo non sia presente.

```
CREATE          DEFINER          =          CURRENT_USER          TRIGGER  
`mydb`.`EsercizioInesist_BEFORE_INSERT_1` BEFORE INSERT ON `Contrassegna` FOR EACH  
ROW
```

```
BEGIN
```

```
    DECLARE esercizio_count INT;
```

```
-- Controlla se l'esercizio esiste nella tabella SchedaEsercizi
```

```
SELECT COUNT(*) INTO esercizio_count
```

```
FROM Composta
```

```
WHERE Utente = NEW.Utente
```

```
    AND DataEmissioneScheda = NEW.DataEmissScheda
```

```
AND Macchinario = NEW.Nome;
```

```
-- Se l'esercizio non esiste, genera un errore
```

```
IF esercizio_count = 0 THEN
```

```
    SIGNAL SQLSTATE '45900'
```

```
    SET MESSAGE_TEXT = 'Esercizio non esiste nella scheda.';
```

```
END IF;
```

```
END
```

OrarioInser_BEFORE_INSERT_1

Trigger creato per controllare che non venga iniziata l'esecuzione di un esercizio nell'orario di chiusura della palestra.

```
CREATE DEFINER = CURRENT_USER TRIGGER `mydb`.`OrarioInser_BEFORE_INSERT_1`  
BEFORE INSERT ON `Contrassegna` FOR EACH ROW
```

```
BEGIN
```

```
    IF HOUR(CURRENT_TIME()) < 7 OR HOUR(CURRENT_TIME()) > 23 THEN
```

```
        SIGNAL SQLSTATE '45001'
```

```
        SET MESSAGE_TEXT = 'L'orario di inserimento deve essere compreso tra le 7:00 e le 23:00';
```

```
    END IF;
```

```
END
```

Contrassegna_BEFORE_UPDATE

Trigger creato per controllare che il numero di serie svolte non sia maggiore del numero di serie indicate sulla scheda.

```
CREATE DEFINER = CURRENT_USER TRIGGER `mydb`.`Contrassegna_BEFORE_UPDATE`  
BEFORE UPDATE ON `Contrassegna` FOR EACH ROW  
  
BEGIN  
  
    DECLARE num_serie INT;  
  
    -- Ottieni il numero di serie dall'esercizischeda  
  
    SELECT NumeroSerie INTO num_serie  
  
    FROM Composta  
  
    WHERE new.Nome = Macchinario  
  
    AND new.DataEmissioneScheda =DataEmissioneScheda  
  
    AND new.Utente = Composta.Utente;  
  
    -- Verifica se la serie corrente è maggiore del numero di serie  
  
    IF new.SerieCorrente > num_serie THEN  
  
        SIGNAL SQLSTATE '45002'  
  
        SET MESSAGE_TEXT = 'La serie corrente è maggiore del numero di serie consentito.';  
  
    END IF;  
  
END
```

OrarioUpdate_BEFORE_UPDATE_1

Trigger creato per controllare che non venga aggiornata l'esecuzione di un esercizio nell'orario di chiusura della palestra.

```
CREATE          DEFINER          =          CURRENT_USER          TRIGGER
`mydb`.`OrarioUpdate_BEFORE_UPDATE_1` BEFORE UPDATE ON `Contrassegna` FOR EACH
ROW

BEGIN

    IF HOUR(CURRENT_TIME()) < 7 OR HOUR(CURRENT_TIME()) > 23 THEN

        SIGNAL SQLSTATE '45003'

        SET MESSAGE_TEXT = 'L\'orario di aggiornamento deve essere compreso tra le 7:00 e le
23:00';

    END IF;

END
```

Esercizio_BEFORE_INSERT

Trigger creato per controllare se il macchinario (Esercizio negli schemi) che si sta andando ad inserire nel database è già presente.

```
CREATE DEFINER = CURRENT_USER TRIGGER `mydb`.`Esercizio_BEFORE_INSERT`
BEFORE INSERT ON `Esercizio` FOR EACH ROW

BEGIN

    DECLARE count_existing_exercises INT;

    -- Controlla se l'esercizio è già presente nel database

    SELECT COUNT(*)

    INTO count_existing_exercises
```

```
FROM Esercizio
```

```
WHERE Nome = NEW.Nome;
```

```
-- Se l'esercizio è già presente, solleva un errore
```

```
IF count_existing_exercises > 0 THEN
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Esercizio già presente nel database.';
```

```
END IF;
```

```
END
```

Eventi

clean_3yoScheda

Tale evento ha la funzione di eliminare tutte le schede create da più di 3 anni e grazie alla configurazione di eliminazione a cascata sulle chiavi esterne si andranno ad eliminare anche tutte le tuple nelle altre entità inerenti a quelle eliminate (sessioni di allenamento svolte con quella determinata scheda, esercizi svolti durante una sessione di allenamento facenti parte di quella determinata scheda, ecc).

```
create event 'clean_3yoSchede'
```

```
on schedule
```

```
    every 1 year
```

```
    starts current_timestamp + interval 3 year
```

```
    on completion preserve
```

```
do
```

```
    delete from 'Scheda' where 'DataEmissione' < (now() - interval 3 year)
```

Viste

Non sono state usate viste.

Stored Procedures e transazioni

aggiungi_esercizio

Procedura che permette l'inserimento di un nuovo Macchinario (Esercizio negli schemi) in palestra.

```
CREATE PROCEDURE `aggiungi_esercizio` (  
    in nome varchar(50) )
```

```
BEGIN
```

```
    insert into `Esercizio` (`Nome`)
```

```
    values (nome);
```

```
END
```

aggiungi_personaltrainer

Procedura che permette l'inserimento di un nuovo Personal Trainer in palestra.

```
CREATE PROCEDURE `aggiungi_personaltrainer` (  
    in cf char(16),
```

```
    in nome varchar(50),
```

```
    in cognome varchar(50),
```

```
    in cognome varchar(50),
```

```
    in username varchar(20),
```

```
    in passwrđ varchar(50) )
```

```
BEGIN
```

declare exit handler for sqlexception

BEGIN

rollback; -- annulla la transazione

resignal; -- ritorna il segnale al client

END;

if(mydb.cf_check(cf) is FALSE) then

signal sqlstate '45001'set message_text= 'CODICE FISCALE ERRATO';

END if;

set transaction isolation level READ COMMITTED; -- read committed perché unrepetable read è ok in questo caso

start transaction;

-- Aggiungo alla tabella del generaluser settando il tipi e di seguito alla tabella dei PT

insert into `GeneralUser` (`Username`, `pass`, `tipo`)

values (username, sha1(passwrđ), 'pt');

insert into `PersonalTrainer` (`CF`, `Nome`, `Cognome`, `Username`)

values (cf, nome, cognome, username);

COMMIT;

END

aggiungi_scheda_esercizio

Procedura per far sì che il Personal Trainer possa aggiungere un esercizio alla Scheda di un Utente.

```
CREATE PROCEDURE `aggiungi_scheda_esercizio` (
```

```
    in numeroserie int,
```

```
    in numeroripetizioni int,
```

```
    in indice int,
```

```
    in nomeesercizio varchar(50),
```

```
    in nomemacchinario varchar(50),
```

```
    in cfuser char(16) )
```

```
BEGIN
```

```
    declare dataemissione date;
```

```
    declare exit handler for sqlexception
```

```
BEGIN
```

```
        rollback; -- annulla la transazione
```

```
    resignal; -- ritorna il segnale al client
```

```
END;
```

```
if(mydb.cf_check(cfuser) is FALSE) then
```

```
    signal sqlstate '45020' set message_text= 'CODICE FISCALE ERRATO';
```

```
END if;
```

set transaction isolation level READ COMMITTED; -- read committed perché unrepetable
read è ok in questo caso

start transaction;

```
select Scheda.DataEmissione into dataemissione  
from Scheda  
where Utente = cfuser and DataScad is null;
```

```
insert `Composta` (`DataEmissioneScheda`, `NumeroSerie`, `NumeroRipetizioni`,  
`Indice`, `Macchinario`, `Utente`, `NomeEsercizio`)  
values (dataemissione, numeroserie, numeroripetizioni, indice, nomemacchinario,  
cfuser, nomeesercizio);  
  
COMMIT;  
  
END
```

aggiungi_utente

Procedura per aggiungere un nuovo utente iscritti in palestra.

```
CREATE PROCEDURE `aggiungi_utente` (  
    in cf char(16),  
    in nome varchar(50),  
    in cognome varchar(50),  
    in username varchar(20),  
    in passwd varchar(50) )  
  
BEGIN
```

```
declare ptcf varchar(20);
```

```
declare exit handler for sqlexception
```

```
BEGIN
```

```
    rollback; -- annulla la transazione
```

```
    resignal; -- ritorna il segnale al client
```

```
END;
```

```
if(mydb.cf_check(cf) is FALSE) then
```

```
    signal sqlstate '45056' set message_text= 'CODICE FISCALE ERRATO';
```

```
END if;
```

set transaction isolation level READ COMMITTED; -- read committed perché unrepetable read è ok in questo caso

```
start transaction;
```

-- Selezione PersonalTrainer con meno utenti e aggiungo nuovo utente al generaluser con il suo tipo

```
select PersonalTrainer.CF into ptcf
```

```
from PersonalTrainer left join Utente on PersonalTrainer.CF = Utente.PersonalTrainer
```

```
group by PersonalTrainer.CF
```

```
order by count(Utente.PersonalTrainer) ASC
```

```
limit 1;
```

```
insert into `GeneralUser` (`Username`, `Pass`, `Tipo`)
```

```
values (username, sha1(passwrđ), 'utente');
```

```
-- Aggiungo utente all'entità
```

```
insert into `Utente` (`CF`, `Nome`, `Cognome`, `Username`, `PersonalTrainer`)
```

```
values(cf, nome, cognome, username, ptcf);
```

```
COMMIT;
```

```
END
```

archivia_scheda

Procedura usata dal Personal Trainer per archiviare la scheda di un utente.

```
CREATE PROCEDURE `archivia_scheda` (
```

```
    in cfuser char(16) )
```

```
BEGIN
```

```
    declare ptcf char(16);
```

```
    declare datascad date;
```

```
    declare exit handler for sqlexception
```

```
BEGIN
```

```
        rollback; -- annulla la transazione
```

```
    resignal; -- ritorna il segnale al client
```

```
END;
```

```
set datascad = curdate();
```

```
IF (mydb.check_sessione(cfuser) = 1) THEN
```

```
    signal sqlstate '45014' set message_text = 'UTENTE STA ESEGUENDO  
UNA SESSIONE CON LA SCHEDA CHE VUOI ARCHIVIARE: ASPETTA CHE FINISCA';
```

```
END IF;
```

```
    set transaction isolation level READ COMMITTED; -- read committed perché unrepetable  
read è ok in questo caso
```

```
start transaction;
```

```
-- Controllo se l'utente è associato correttamente al personal trainer
```

```
select PersonalTrainer.CF into ptcf
```

```
from Utente
```

```
join PersonalTrainer on Utente.PersonalTrainer = PersonalTrainer.CF
```

```
where Utente.CF = cfuser;
```

```
if ptcf is null then
```

```
    SIGNAL SQLSTATE '45000'
```

```
    SET MESSAGE_TEXT = 'Utente selezionato non è associato a nessun personal trainer.';
```

```
END IF;
```

```
-- Controllo se esiste una scheda attiva per l'utente
```

```
if not exists (
```

```
select *  
  
from Scheda  
  
where Utente = cfuser and Tipo = 'attiva'  
  
) then  
  
    SIGNAL SQLSTATE '45000'  
  
    SET MESSAGE_TEXT = 'Non esiste una scheda attiva per utente selezionato.';  
  
END if;
```

```
        update Scheda  
  
        set DataScad = datascad  
  
        where Utente = cfuser and Tipo = 'attiva';
```

```
        update Scheda  
  
        set Tipo= 'scaduta'  
  
        where Utente = cfuser and Tipo = 'attiva';
```

```
COMMIT;
```

```
END
```

cf_check

Funzione per controllare l'esatta immissione di un formato Codice Fiscale.

```
CREATE FUNCTION `cf_check` (CF char(16)) returns boolean
```

deterministic

BEGIN

if CF regexp '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]\$\n' then

return TRUE;

else

return FALSE;

end if;

END

cf_pt

Procedura per prendere il codice fiscale di un personal trainer.

CREATE PROCEDURE `cf\_pt` (

in username varchar(20)

)

BEGIN

SET TRANSACTION ISOLATION LEVEL READ COMMITTED ;

START TRANSACTION ;

SELECT CF

FROM PersonalTrainer

```
WHERE PersonalTrainer.Username = username;
```

```
COMMIT;
```

```
END
```

cf_user

Procedura usata per prendere il codice fiscale di un utente.

```
CREATE PROCEDURE `cf_user` (
```

```
    in username varchar(20)
```

```
)
```

```
BEGIN
```

```
    SET TRANSACTION ISOLATION LEVEL READ COMMITTED ;
```

```
    START TRANSACTION ;
```

```
        SELECT CF
```

```
        FROM Utente
```

```
        WHERE Utente.Username = username;
```

```
COMMIT;
```

```
END
```

crea_scheda

Procedura usata dal personal trainer per creare una scheda ad un utente.

```
CREATE PROCEDURE `crea_scheda` (
```

```
    in data_emissione date,
```

```
    in cfuser char(16),
```

```
    in cfpt char(16) )
```

```
BEGIN
```

```
    declare tempcfpt char(16);
```

```
    declare exit handler for sqlexception
```

```
BEGIN
```

```
        rollback; -- annulla la transazione
```

```
    resignal; -- ritorna il segnale al client
```

```
END;
```

```
if((mydb.cf_check(cfuser) is FALSE) OR mydb.cf_check(cfpt)is FALSE) then
```

```
    signal sqlstate '45006'set message_text= 'CODICE FISCALE ERRATO';
```

```
END if;
```

```
        set transaction isolation level READ COMMITTED;

start transaction;


        select PersonalTrainer into tempcfpt

from Utente

where CF= cfuser;


if(tempcfpt <> cfpt) then

        signal sqlstate '45007' set message_text= 'CLIENTE NON ASSOCIATO A
QUESTO PERSONAL TRAINER';

        END if;


        insert into `Scheda`(`DataEmissione`, `DataScad`, `Tipo`, `Utente`)

values (data_emissione, NULL, 'attiva', cfuser);


        COMMIT;


END
```

esegui_esercizio

Procedura usata dall'utente per iniziare l'esecuzione di un esercizio.

```
CREATE PROCEDURE `esegui_esercizio` (

        in nome varchar(50),

        in cfuser char(16) )
```

BEGIN

declare dataallen date;

declare dataemiss date;

declare nomeeserc varchar(50);

declare exit handler for sqlexception

BEGIN

rollback; -- annulla la transazione

resignal; -- ritorna il segnale al client

END;

if(mydb.cf_check(cfuser) is FALSE) then

signal sqlstate '45100'set message_text= 'CODICE FISCALE ERRATO';

END if;

set transaction isolation level READ UNCOMMITTED; -- read committed perché
unrepeatable read è ok in questo caso

start transaction;

select Scheda.DataEmissione into dataemiss

from Scheda

where Utente = cfuser and DataScad is null;

select SessioneAllenamento.DataAllenamento into dataallen

from SessioneAllenamento

where cfuser = Utente and OrarioFine is null

limit 1;

select Esercizio.Nome into nomeeserc

from Esercizio

where Nome = nomeeserc;

insert into `Contrassegna`(`Nome`, `DataAllen`, `SerieCorrente`, `Utente`,
`DataEmissScheda`)
values (nome, dataallen, 1, cfuser, dataemiss);

COMMIT;

END

esegui_serieeserc

Procedura usata dall'utente per incrementare il numero di serie svolte di un dato esercizio.

```
CREATE PROCEDURE `esegui_serieeserc` (  
    in cfuser char(16),  
    in nome varchar(50)  
)
```

```
BEGIN
```

```
        declare data_allenamento date;

declare serie_corrente int;

declare nuova_serie int;

declare exit handler for sqlexception

BEGIN

        rollback; -- annulla la transazione

        resignal; -- ritorna il segnale al client

        END;

if(mydb.cf_check(cfuser) is FALSE) then

        signal sqlstate '45200' set message_text= 'CODICE FISCALE ERRATO';

        END if;


        set transaction isolation level READ UNCOMMITTED; -- read committed perché unrepetable
read è ok in questo caso

        start transaction;


        -- Trova la data di allenamento della sessione corrente

        SELECT DataAllenamento INTO data_allenamento

        FROM SessioneAllenamento

        WHERE Utente = cfuser AND OrarioFine IS NULL

        LIMIT 1;


        -- Controlla se l'utente ha una sessione di allenamento attiva

        IF data_allenamento IS NULL THEN
```

```
SIGNAL SQLSTATE '45321'

SET MESSAGE_TEXT = 'L"utente non ha avviato una sessione di allenamento attiva.';

END IF;

-- Controlla se l'esercizio esiste già in Contrassegna per la data di allenamento corrente

SELECT SerieCorrente INTO serie_corrente

FROM Contrassegna

WHERE Contrassegna.Nome = nome

        AND DataAllen = data_allenamento

        AND Utente = cfuser

LIMIT 1;

-- set nuova_serie = serie_corrente + serie;

set nuova_serie = serie_corrente + 1;

-- L'esercizio esiste già in Contrassegna, quindi aggiorna il numero di serie svolte

UPDATE Contrassegna

SET SerieCorrente = nuova_serie

WHERE Contrassegna.Nome = nome

        AND DataAllen = data_allenamento

        AND Utente = cfuser;

COMMIT;
```

END

fine_allenamento

Procedura usata dall'utente per concludere la sessione di allenamento.

```
CREATE PROCEDURE `fine_allenamento` (
```

```
    in cfuser char(16)
```

```
)
```

```
BEGIN
```

```
    declare dataallenamento date;
```

```
    declare ora time;
```

```
        declare exit handler for sqlexception
```

```
BEGIN
```

```
            rollback; -- annulla la transazione
```

```
        resignal; -- ritorna il segnale al client
```

```
        END;
```

```
if(mydb.cf_check(cfuser) is FALSE) then
```

```
    signal sqlstate '45004' set message_text= 'CODICE FISCALE ERRATO';
```

```
END if;
```

```
set ora = curtime();
```

```
set transaction isolation level READ COMMITTED;
```

```
start transaction;
```

```
update SessioneAllenamento
```

```
set OrarioFine = ora
```

```
where cfuser= Utente and OrarioFine is null;
```

```
COMMIT;
```

```
END
```

generareport

Procedura usata dal Personal Trainer per generare un resoconto dell'andamento degli utenti a lui associati.

```
CREATE PROCEDURE `generareport`(  
IN var_trainer_cf VARCHAR(20),  
IN var_dataInizio DATE,  
IN var_dataFine DATE)  
BEGIN
```

```
declare exit handler for sqlexception ## Dichiarazione Gestore eccezione sollevata dal  
Trigger
```

```
begin
```

```
rollback; ## Annullamento Transazione  
resignal; ## Ridirezione Segnale al Client  
end;
```

```
DROP TABLE IF EXISTS report_temp;
```

```
-- Crea una tabella temporanea per il report
```

```
CREATE TEMPORARY TABLE report_temp (
```

```
CF VARCHAR(20),
```

```
Nome VARCHAR(45),
```

```
Cognome VARCHAR(45),
```

```
DataScheda DATE,
```

```
Sessione DATE,
```

```
Completamento VARCHAR(10),
```

```
Durata CHAR(10));
```

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

```
START TRANSACTION;
```

```
-- Inserisci i dati nel report temporaneo
```

```
INSERT INTO report_temp (CF, Nome, Cognome, DataScheda, Sessione,  
Completamento, Durata)
```

```
SELECT u.CF as CF, u.Nome as Nome, u.Cognome as Cognome, sa.DataEmissione  
as DataScheda, ses.DataAllenamento as Sessione,
```

```
CONCAT(IFNULL((SUM(co.SerieCorrente) / SUM(comp.NumeroSerie)) *  
100, 0), '%%') AS Completamento,
```

```
CAST(TIMEDIFF(ses.OrarioFine, ses.OrarioInizio) AS CHAR(10)) AS  
Durata
```

```
FROM Utente u
```

```
JOIN Scheda sa ON u.CF = sa.Utente
```

```
JOIN SessioneAllenamento ses ON sa.Utente = ses.Utente
```

```
AND sa.DataEmissione = ses.DataEmiss
```

```
JOIN Composta comp ON sa.Utente = comp.Utente
```

```
AND sa.DataEmissione = comp.DataEmissioneScheda
```

```
LEFT JOIN Contrassegna co ON ses.Utente = co.Utente
```

```
AND ses.DataEmiss = comp.DataEmissioneScheda
```

```
AND ses.DataAllenamento = co.DataAllen
```

```
AND comp.Macchinario = co.Nome
```

```
WHERE u.PersonalTrainer = var_trainer_cf and OrarioFine is not NULL
```

```
AND ses.DataAllenamento BETWEEN var_dataInizio AND var_dataFine
```

```
GROUP BY u.CF, u.Nome, u.Cognome, ses.DataAllenamento, sa.DataEmissione;
```

```
-- Seleziona i dati dal report temporaneo per visualizzarli
```

```
SELECT *
```

```
FROM report_temp;
```

```
COMMIT;
```

```
END
```

inizio_allenamento

Procedura usata dall'utente per avviare una sessione di allenamento.

```
CREATE PROCEDURE `inizio_allenamento` (  
    in cfuser char(16) )  
  
BEGIN  
  
    declare dataemissione date;  
  
    declare giorno date;  
  
    declare ora time;  
  
  
    declare exit handler for sqlexception  
BEGIN  
        rollback; -- annulla la transazione  
        resignal; -- ritorna il segnale al client  
    END;  
  
    set transaction isolation level READ COMMITTED;  
  
    start transaction;  
  
  
    set giorno = curdate();  
  
    set ora = curtime();  
  
  
    if(mydb.cf_check(cfuser) is FALSE) then
```

```
        signal sqlstate '45003' set message_text= 'CODICE FISCALE ERRATO';  
    END if;
```

```
        select Scheda.DataEmissione into dataemissione  
    from Scheda  
    where DataScad is null and Utente = cfuser;
```

```
        insert into `SessioneAllenamento` ( `DataAllenamento`, `OrarioInizio`, `OrarioFine`,  
    `Utente`, `DataEmiss`)  
    values (giorno, ora, null, cfuser, dataemissione);
```

```
    COMMIT;
```

```
END
```

login

Procedura usata per effettuare l'accesso all'applicazione.

```
CREATE PROCEDURE `login` (  
    IN username VARCHAR(20),  
    IN varpassword VARCHAR(50),  
    OUT varrole INT)  
BEGIN  
  
    DECLARE temprole varchar(45) ;
```

```
SELECT Tipo into temprole  
FROM GeneralUser  
WHERE GeneralUser.Username = username AND Pass = SHA1(varpassword)  
LIMIT 1;
```

```
IF temprole = "pt" THEN  
    SET varrole = 1 ;  
ELSEIF temprole = "utente" THEN  
    SET varrole = 2 ;  
ELSEIF temprole = "segreteria" THEN  
    SET varrole = 3 ;  
ELSE  
    SET varrole = 4 ;  
END IF ;
```

```
COMMIT;
```

```
END
```

pt_visualizzascheda

Procedura usata dal Personal Trainer per poter visualizzare la scheda attiva di un determinato utente a lui associato.

```
CREATE PROCEDURE `pt_visualizzascheda` (  
    in cfpt char(16),
```

```
in cfuser char(16)
)
BEGIN

    DECLARE ptcfcontrol CHAR(16);

    declare tempcfpt char(16);

    declare exit handler for sqlexception
    begin

        rollback;

        resignal;

    end;

    if(mydb.cf_check(cfuser) is FALSE) and (mydb.cf_check(cfpt) is FALSE) then

        signal sqlstate '45039' set message_text= 'CODICE FISCALE ERRATO';

    END if;

    set transaction isolation level READ UNCOMMITTED;

    start transaction;

    -- Verifica se l'utente è associato al personal trainer

    SELECT CF INTO ptcfcontrol

    FROM PersonalTrainer

    WHERE CF = cfpt;
```

```
IF cfpt IS NULL THEN
```

```
    SIGNAL SQLSTATE '45243' SET MESSAGE_TEXT = 'Personal Trainer non trovato.';
```

```
END IF;
```

```
select PersonalTrainer into tempcfpt
```

```
    from Utente
```

```
where CF= cfuser;
```

```
if(tempcfpt <> cfpt) then
```

```
    signal sqlstate '45077' set message_text= 'CLIENTE NON ASSOCIATO A  
QUESTO PERSONAL TRAINER';
```

```
END if;
```

```
-- Verifica se per l'utente esiste una scheda attiva
```

```
IF NOT EXISTS (
```

```
    SELECT 1
```

```
    FROM Scheda
```

```
    WHERE Utente = cfuser AND Tipo = 'attiva'
```

```
) THEN
```

```
    SIGNAL SQLSTATE '45355' SET MESSAGE_TEXT = 'Nessuna scheda attiva trovata  
collegata a questo utente.';
```

```
END IF;
```

```
-- Se entrambi i controlli sono passati, restituisci gli elementi della scheda attiva dell'utente
```

```
SELECT DataEmissione, s.Utente , NumeroSerie, NumeroRipetizioni, Indice, Macchinario,  
NomeEsercizio
```

```
FROM Scheda s join Composta c on s.Utente = c.Utente
```

```
WHERE s.Utente = cfuser AND Tipo = 'attiva' and DataEmissioneScheda = DataEmissione
```

```
Order by Indice;
```

```
COMMIT;
```

```
END
```

visualizza_clienti

Procedura usata dal Personal Trainer per poter visionare gli utenti a lui associati.

```
CREATE PROCEDURE `visualizza_clienti` ( -- pt visualizza i clienti a lui associati
```

```
in cfpt char(16) )
```

```
BEGIN
```

```
declare ptcfcontrol char(16);
```

```
declare exit handler for sqlexception
```

```
BEGIN
```

```
rollback; -- annulla la transazione
```

```
resignal; -- ritorna il segnale al client
```

```
END;
```

```
if(mydb.cf_check(cfpt) is FALSE) then
```

```
signal sqlstate '45039' set message_text= 'CODICE FISCALE ERRATO';
```

```
END if;
```

```
set transaction isolation level READ COMMITTED;  
  
start transaction;
```

```
select CF, Nome, Cognome  
  
from Utente  
  
where PersonalTrainer = cfpt;
```

```
COMMIT;
```

```
END
```

visualizza_esercizi

Procedura usata dal Personal Trainer per poter visionare i macchinari disponibili in palestra.

```
CREATE PROCEDURE `visualizza_esercizi` ()
```

```
BEGIN
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;
```

```
SELECT Nome  
  
FROM Esercizio;
```

```
COMMIT;
```

```
END
```

visualizza_esercizimancanti

Procedura usata dall'utente per poter visionare gli esercizi non ancora iniziati della sua scheda in durante una sessione di allenamento.

```
CREATE PROCEDURE `visualizza_esercizimancanti` (
```

```
    in cfuser char(16)
```

```
)
```

```
BEGIN
```

```
    declare data_attuale DATE;
```

```
    declare data_scheda DATE;
```

```
    set data_attuale = CURDATE();
```

```
    if(mydb.cf_check(cfuser) is FALSE) then
```

```
        signal sqlstate '45039' set message_text= 'CODICE FISCALE ERRATO';
```

```
    END if;
```

```
    set transaction isolation level READ COMMITTED;
```

```
    start transaction;
```

```
    -- Trova la data della scheda attiva dell'utente
```

```
    SELECT DataEmissione INTO data_scheda
```

```
FROM Scheda
```

```
WHERE Utente = cfuser AND Tipo = 'attiva';
```

```
-- Restituisci gli esercizi mancanti
```

```
SELECT c.Macchinario, c.NumeroSerie, c.NumeroRipetizioni, c.Indice
```

```
FROM Composta c
```

```
LEFT JOIN Contrassegna co ON c.Macchinario = co.Nome
```

```
AND co.DataAllen = data_attuale
```

```
AND co.Utente = cfuser
```

```
WHERE c.DataEmissioneScheda = data_scheda
```

```
AND co.Nome IS NULL
```

```
ORDER BY c.Indice;
```

```
COMMIT;
```

```
END
```

visualizza_schedaattiva

Procedura usata dall'utente per poter visionare la sua attuale scheda attiva.

```
CREATE PROCEDURE `visualizza_schedaattiva` (
```

```
in cfuser char(16) )
```

```
BEGIN
```

```
declare dataemi date;
```

```
declare exit handler for sqlexception
```

```
begin
```

```
    rollback;
```

```
    resignal;
```

```
end;
```

```
if(mydb.cf_check(cfuser) is FALSE) then
```

```
    signal sqlstate '45300' set message_text= 'CODICE FISCALE ERRATO';
```

```
END if;
```

set transaction isolation level READ COMMITTED; -- read committed perché unrepetable read è ok in questo caso

```
start transaction;
```

```
select Scheda.DataEmissione into dataemi
```

```
from Scheda
```

```
where Utente = cfuser and DataScad is null;
```

```
if dataemi is null then
```

```
    signal sqlstate '45450' set message_text= 'nessuna scheda attiva esistente';
```

```
end if;
```

```
select DataEmissioneScheda, NumeroSerie, NumeroRipetizioni, Indice, Macchinario,  
NomeEsercizio
```

```
from Composta
```

where DataEmissioneScheda = dataemi and Utente = cfuser

order by Indice;

COMMIT;

END

visualizza_schedearchivate

Procedura usata dall'utente per poter visionare la data di emissione e scadenza di tutte le schede archiviate che ha.

```
CREATE PROCEDURE `visualizza_schedearchivate` (
```

```
    in cfuser char(16)
```

```
)
```

```
BEGIN
```

```
    declare numsched int;
```

```
    if(mydb.cf_check(cfuser) is FALSE) then
```

```
        signal sqlstate '45334' set message_text= 'CODICE FISCALE ERRATO';
```

```
    END if;
```

set transaction isolation level READ COMMITTED; -- read committed perché unrepetable read è ok in questo caso

```
start transaction;
```

```
SELECT count(*) into numsched

FROM Sceda

WHERE DataScad is not NULL and Utente = cfuser;

IF(numsched = 0) THEN

    signal sqlstate '45025' set message_text = 'NON HAI SCHEDE
ARCHIVIAATE';

END IF;

SELECT DataEmissione, DataScad

FROM Sceda

WHERE Utente = cfuser AND Tipo = 'scaduta';

COMMIT;

END
```

visualizza_seriemancanti

Procedura usata dall'utente per poter visionare quante serie mancano per l'esecuzione di un dato esercizio.

```
CREATE PROCEDURE `visualizza_seriemancanti` (

    IN cfuser CHAR(16),
```

```
IN nomeese VARCHAR(50)
)
BEGIN
    DECLARE data_emissione_scheda DATE;

    DECLARE serie_corrente INT;

    DECLARE serie_totali INT;

    DECLARE serie_mancanti INT;

    if(mydb.cf_check(cfuser) is FALSE) then
        signal sqlstate '45569' set message_text= 'CODICE FISCALE ERRATO';
    END if;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    start transaction;

    -- Trova la data di emissione della scheda attiva dell'utente

    SELECT DataEmissione INTO data_emissione_scheda

    FROM Scheda

    WHERE Utente = cfuser AND Tipo = 'attiva';

    -- Trova il valore "seriecorrente" per l'esercizio durante la sessione di allenamento corrente

    SELECT SerieCorrente

    INTO serie_corrente
```

```
FROM Contrassegna
```

```
WHERE Nome = nomeese
```

```
AND DataAllen = CURDATE()
```

```
AND Utente = cfuser;
```

```
if(serie_corrente is null) then
```

```
    set serie_corrente = 0;
```

```
end if;
```

```
-- Trova il numero totale di serie previste per l'esercizio
```

```
SELECT NumeroSerie INTO serie_totali
```

```
FROM Composta
```

```
WHERE DataEmissioneScheda = data_emissione_scheda
```

```
AND Macchinario = nomeese;
```

```
-- Calcola il numero di serie mancanti
```

```
SET serie_mancanti = serie_totali - serie_corrente;
```

```
-- Restituisce il numero di serie mancanti
```

```
SELECT serie_mancanti AS SerieMancanti;
```

```
COMMIT;
```

```
END
```

visualizza_singolaarchiviata

Procedura usata dall'utente per poter visionare gli esercizi, ripetizioni e serie inerenti di una sua scheda archiviata.

```
CREATE PROCEDURE `visualizza_singolaarchiviata` (  
    in dataemissione date,  
    in cfuser char(16) )  
BEGIN  
  
    declare numese int;  
  
    declare exit handler for sqlexception  
    begin  
        rollback;  
        resignal;  
    end;  
  
    if(mydb.cf_check(cfuser) is FALSE) then  
        signal sqlstate '45322' set message_text= 'CODICE FISCALE ERRATO';  
    END if;  
  
    set transaction isolation level READ COMMITTED; -- read committed perché unrepetable read è  
    ok in questo caso  
  
    start transaction;
```

```
SELECT COUNT(*) INTO numese
```

```
FROM Scheda s

JOIN Composta c ON s.DataEmissione = c.DataEmissioneScheda AND s.Utente =
c.Utente

WHERE s.Utente = cfuser AND s.DataEmissione = dataemissione;

IF numese = 0 THEN

    signal sqlstate '45021' set message_text = 'DATA NON NEL DATABASE';

ELSE

    SELECT    DataEmissione,    NumeroSerie,    NumeroRipetizioni,    Indice,
Macchinario, NomeEsercizio

    FROM Scheda s

    JOIN Composta c ON s.DataEmissione = c.DataEmissioneScheda AND
s.Utente = c.Utente

    WHERE s.Utente = cfuser AND s.DataEmissione = dataemissione;

END IF;

COMMIT;

END
```